計算システム第２レジュメ vol.1

2026 年 10 月 20 日

本講義ではソフトウェアがメインの内容になっている。
成績は「期末（公式カンペあり）＋小テスト（Utol 上で行うので講義資料を見ても良い)」で行うらしい
小テストは現地にいかないといけないのでそれ以外は出席しなくても良い

目次

1 OS の役割 2
1.1 OS とは . . . 2
1.2 OS の役割 . . . 3
1.3 OS の変遷と形態 . . . 4
1.4 OS の階層構造と構成要素 . . . 5
1.5 OS の処理単位とモジュール構成 . . . 6
1.6 OS の構成方式 . . . 7
1.7 OS の起動と中断・終了 . . . 8

2 プロセスの管理とスケジューリング 10
2.1 プロセスの管理機能 . . . 10
2.2 プロセスの状態と状態遷移 . . . 11
2.3 プロセス制御ブロックとその管理 . . . 12
2.4 スレッドとその利用場面 . . . 14
2.5 プロセスのスケジューリング . . . 15
2.6 スケジューリングアルゴリズム . . . 17

3 ネットワークとサイバーセキュリティ 20
3.1 サイバーセキュリティの基本概念と背景 . . . 20
3.2 東京大学におけるセキュリティ教育 . . . 21
3.3 サイバー攻撃の現状と構造 . . . 22
3.4 ネットワークとプロトコルの基礎 . . . 24
3.5 攻撃者によるネットワーク偵察の手法 . . . 26
3.6 東京大学（UTNET）のネットワークとセキュリティ対策 . . . 27
3.7 関連技術（余談） . . . 28
3.8 まとめ . . . 29

1　OS の役割

1.1　OS とは

1.1.1　コンピュータシステムの構成要素と設計

- **ハードウェア（高速性の実現）**：CPU，HDD，キーボード，ディスプレイなど
- **ソフトウェア（適応性の実現）**：ブラウザ，Office，音楽プレーヤなどのアプリケーション
- **システム設計の課題**：
  - ハードウェアとソフトウェアの機能分担およびトレードオフの決定
  - 役割分担と、双方の機能を相互利用する仕組みの構築

1.1.2　OS（Operating System）の定義と役割

- 計算機ハードウェアとソフトウェアを橋渡しする基盤システムソフトウェア
- ソフトウェア（ユーザ）とハードウェアの間に位置し、計算機資源を効率よく管理・利用しやすい環境を提供
- 現代のコンピュータシステムに不可欠な基盤（縁の下の力持ち）
- **OS の 2 大原理（管理と保護）**：
  - 物理ハードウェア機構の隠蔽（抽象化）
  - 共用ハードウェア資源の管理・制御
  - 実行時におけるソフトウェアやシステムの保護

1.1.3　実行モードによるアクセス制御

- **カーネルモード（特権モード）**：ハードウェアへのアクセスと命令実行が無制限に可能
- **ユーザモード**：システム保護のため、実行できる命令に制限を設置
  - 権限に応じて適切にハードウェアや OS を操作できるよう明確に区分けする
- **二面性**：
  - ハードウェア視点：HW を操作・制御するプログラム
  - ソフトウェア視点：HW を抽象化する仮想計算機

まとめ

OS はハードウェアとソフトウェアの間に立ち、複雑な物理機構を隠蔽・抽象化しながら資源の「管理」と「保護」を担う基盤ソフトウェアです。特権的なカーネルモードと制限付きのユーザモードを分けることで、システムの安定性とセキュリティを確保しています。

例え話による解説

OS を「レストランの総支配人」に例えてみましょう。

- **ハードウェア（厨房と調理器具）**：高速で料理（処理）を作りますが，客（アプリ）が直接操作すると危険です．
- **ソフトウェア（客）**：自由な注文を出しますが，厨房の詳しい仕様を知る必要はありません．
- **OS（支配人）**：客からの注文を取り，厨房の器具（ハードウェア）を安全に差配・管理します．絶対に触らせない「厨房（カーネルモード）」と，客が過ごす「客席（ユーザモード）」を区別することで，店全体の安全と円滑な運営を守っています．

1.2 OS の役割

1.2.1 ハードウェアの抽象化と仮想化

- **仮想計算機の提供**：ソフトウェアから見て、ハードウェア構成要素をプロセスやファイルなどの概念に抽象化し、それらに対する操作を提供します。
- **ハードウェアの画一化・隠蔽**：多種多様なハードウェア（例：NEC、TOSHIBA、DELL の PC など）の違いを吸収し、ソフトウェア（ブラウザ、Office など）との橋渡しを行う基盤となります。
- **仮想化技術**：実際には存在しない機構が機能しているように見せかけます。
  - ソフトウェアによる実現（VirtualBox、Parallels など）や、カーネル内の支援機能（KVM、Hyper-V など）があります。
  - **効果**：プロセッサやメモリなどの相違を吸収し、プログラムの移植性・互換性を確保。どこでも実行可能になるため、クラウドコンピューティングの基盤にもなります。

1.2.2 資源（リソース）の管理と多重化

- **資源管理者（制御プログラム）**：ハードウェアから見て、CPU、主記憶、ディスクなどへのアクセスを効率的に制御するプログラム群です。
- **多重化**：限られたハードウェア資源を分割して共用します。
  - **時間的多重化**：時間を分割して資源を使用（例：1 つの CPU で複数アプリを実行）。
  - **空間的多重化**：領域を分割して資源を使用（例：メモリ上に複数の異なるデータを格納）。
- **スケジューリングと同期制御**：いつ、誰に、どれだけ資源を使用させるかを決定し、要求の競合を解決して公平に操作させます。

1.2.3 システムの保護

- **脅威からの保護**：
  - **外部からの脅威**：ウイルス、ワーム、クラッキングなど。
  - **内部からの脅威**：他ユーザの悪意ある／誤った操作、バッファオーバーフローなど。
- **達成すべき目標**：守秘性（暴露されない）、完全性（間違いがない）、可用性（使える）、検証可能性（後で分析できる）。
- **最小特権の原則**：

- 処理を実行するために「ちょうど十分なだけの権限」を与えます。
- 適切なアクセス権限／実行権限の付与（誰が、何に対して、何をできるか）。
- 例：アクセス許可（read, write, execute）、許可対象（user, group, others, all）。

まとめ

OS の主な役割は、ハードウェアの複雑さを「抽象化・仮想化」して使いやすくすること、限られたリソースを効率よく分配する「資源管理（多重化）」、そして内外の脅威や誤操作からシステムを守る「保護」です。最小特権の原則に基づくアクセス制御により、安全性を担保しています。

例え話による解説

OS の役割を**「ホテルの運営」**に例えてみましょう。

- **仮想化（抽象化）**：客（ソフトウェア）はボイラー室や配管（ハードウェア）を意識せず、蛇口をひねるだけでお湯が出ます。
- **資源管理（多重化）**：限られた部屋やレストランの席（CPU やメモリ）を、フロント（スケジューラ）が効率よく宿泊客に割り当て、ダブルブッキングを防ぎます。
- **保護とアクセス制御**：客には自分の部屋のカードキー（最小特権）だけを渡し、他人の部屋や従業員専用エリア（カーネルモード）には入れないようにして、全体の安全を守ります。

1.3 OS の変遷と形態

1.3.1 OS の進化と変遷

- **処理方式の進化**：バッチ（一括処理）型から、TSS（Time Sharing System：時分割）型、そして対話型の OS へと発展しました。
- **ハードウェアと技術の発展**：
  - アドレス空間の増大（8 ビット → 16 → 32 → 64 ビット）
  - プログラミング言語の高級化と発展
  - 入出力装置のマルチメディア化および通信機能・モバイル化の充実
- **ユーザインタフェース（UI）の変遷**：
  - TSS 時代のテキストベースから、GUI（グラフィカル）の一般化・マルチメディア化へ移行しました。
  - 入力デバイスの進化：パンチカード → キーボード → マウス → タッチディスプレイ。
- **コンピュータの小型化と用途の属人化**：
  - 軍事用途 → 産業用途 → 家庭用 → 家電用途 → 個人持ち（スマートフォン）へと劇的に小型化・普及しました。
  - 現在では、一人で複数台の「OS」を所有・携帯する時代になっています。

1.3.2 OS の多様な形態（用途別の分類）

- **大型計算機向け OS**：
  - **用途**：データセンターなどで、大量のデータが必要なジョブを高並列で処理することに特化してい

ます。
  - 例： Linux, OS/390
- **サーバ向け OS：**
  - 用途：ネットワーク越しに複数ユーザが共有する資源を管理します。
  - 例： Solaris, FreeBSD, Linux, Windows Server, Mac Server
- **パーソナルコンピュータ（PC）向け OS：**
  - 例： Linux, FreeBSD, Windows 7/8/10, Mac OS X
- **スマートフォン向け OS：**
  - 例： Android, iOS, Windows Embedded/10
- **組込み（家電・機器）向け OS：**
  - 用途：電子レンジ、テレビ、自動車など、専用のソフトウェアのみを実行するために最適化された OS です。
  - 例： Embedded Linux, VxWorks, ITRON, AUTOSAR

まとめ

OS は時代とともに「より使いやすく（UI の進化）」「より身近に（小型化・属人化）」発展してきました。現在では、データセンターの巨大なサーバから、手元のスマートフォン、さらには生活家電や自動車（組込み OS）に至るまで、用途やハードウェアの規模に応じた多様な形態の OS が私たちの生活を支えています。

例え話による解説

OS の多様性を「**乗り物**」に例えてみましょう。

- **大型計算機向け OS（大型貨物列車）：**一度に大量の荷物（データ）を運ぶ（処理する）ことに特化しています。
- **サーバ向け OS（路線バス）：**多くの乗客（ユーザ）を乗せ、ネットワークという道路を使って安全にリソースを提供します。
- **PC・スマホ向け OS（自家用車・自転車）：**個人が自由に、直感的な操作（GUI やタッチパネル）で目的地へ向かう（作業する）ための乗り物です。
- **組込み向け OS（エレベーター）：**「上下に移動する」という特定の目的（専用ソフト）だけを、確実かつ安全に実行します。

1.4 OS の階層構造と構成要素

1.4.1 システムを構成する主な要素

- **ユーザプログラム：**一般ユーザが利用する応用プログラム（アプリケーション）.
- **ミドルウェア：** OS とプログラムの中間に位置し，特定の用途（TCP/IP ネットワークやデータベース管理など）の機能を提供するソフトウェア.
- **ライブラリ：**汎用的なソフトウェアの集合であり，カーネルの代表的な操作群をまとめたもの.
- **ユーザインタフェース：**ユーザプログラムと OS とをつなぐ橋渡し役.

- **カーネル：** OS の基本的かつ中核的な機能を提供する部分（狭義の OS）.
- **デバイスドライバ：**ハードウェア（デバイス）を直接制御するソフトウェア.

1.4.2 プログラムから OS へのアクセス経路

- **API（Application Programming Interface）：**プログラム同士が互いにやり取りするためのインタフェース仕様．ライブラリよりも汎用的な開発環境として提供される.
- **システムコール：**プログラムから OS（カーネル）へのサービス機能の要求.
  - システムコールによる要求は**カーネルモード**で実行される（モノリシックカーネルの場合は全て）.
- **処理の流れ：**プログラムからの要求は，**API** → **ライブラリ** → **システムコール**の順に OS へ伝達される.

1.5 OS の処理単位とモジュール構成

1.5.1 処理単位：プロセス（タスク）

- **プロセスの定義：** CPU で実行されている「動作中のプログラム」と，それが使用する「データの集まり」．計算機システム内部の各種活動に対応する.
- **動的な管理：**プログラム自体はストレージ上の「静的」な存在（データ）だが，実行されてプロセスになると「動的」に管理され，CPU のスケジューリング対象となる.
- 同じプログラムを複数同時に起動した場合，それぞれは**別々の独立したプロセス**として管理される.

1.5.2 OS のモジュール構成とその要件

- 大規模なソフトウェアである OS は，特定の機能を実現するためのデータと手続き（プログラム）の集まりである **OS モジュール**群として構成される.
- **OS モジュールの 3 要件：**
  - **(A) 情報の隠蔽：**インタフェースや仕様のみを公開し，内部構造を隠すことで各モジュールの独立性を高める.
  - **(B) 方針と機構の分離：**「何をするか（方針: policy）」と「どうやって処理するか（機構: mechanism）」を分けることで，柔軟性と独立性を高める.
  - **(C) 階層化：**ユーザからハードウェアに至る機能をレベルごとに階層分けし，モジュール間の通信を局所化・限定する.

1.5.3 カーネルの主な機能群（8 つの主要モジュール）

1. **割込み制御：**事象の発生（キーボード入力や指定時間経過など）に即座に対応する.
2. **入出力制御：**入出力装置の管理と仮想化を行い，異常動作から保護する.
3. **タイマ管理：**ハードウェアのタイマ機構を利用し，現在時刻の管理や時間経過の監視・通知を行う.
4. **メモリ管理：**主記憶（メモリ）を効率的に利用し，プログラムやデータをいつ・どこに配置するかを決定する.
5. **CPU スケジューラ：**ディスパッチャなどの機能を用いて，CPU にプロセスを割り付ける.

6. **プロセス管理**：プロセスの生成から消滅までを管理・スケジュールする．
7. **同期・通信制御**：複数のプログラムが共有資源を矛盾なく扱えるように，協調的動作（同期や連絡）を制御する．
8. **ファイルシステム**：記憶領域の管理，ファイル単位のアクセス提供，およびアクセス制限による保護を行う．

まとめ

OS は，ユーザプログラムから API・ライブラリ・システムコールという「階層」を通じて安全にカーネルへ要求を伝える構造になっています．また，実行中のプログラムである「プロセス」を処理の基本単位とし，カーネル内の様々な「モジュール（メモリ管理や CPU スケジューラなど）」が連携して，複雑なシステム全体を効率よく制御しています．

例え話による解説

OS の階層とプロセスを**「レストランの厨房と料理」**に例えてみましょう．

- **API とシステムコール（階層構造）**：
  - **API / ライブラリ**＝メニュー表．客（プログラム）はメニューから「カレー」を選びます．
  - **システムコール**＝ウェイター．客の注文を安全に厨房（カーネル）へ伝えます．客が直接厨房に入って火を使うことはできません．
- **プログラムとプロセス**：
  - **プログラム**＝レシピ本（静的なデータ）．本棚に置いてあるだけでは何も起きません．
  - **プロセス**＝実際に調理中のカレー（動的な活動）．同じレシピ本を見ながら，3 人のシェフが別々のコンロで 3 人分のカレーを作っている場合，これらは「3 つの別々のプロセス」として独立して管理されます．

1.6 OS の構成方式

1.6.1 代表的なカーネル構成方式

- **モノリシックカーネル（単層カーネル）**
  - カーネルのサービス機能のすべてを，カーネルモードで実行（”広義”のカーネルとして実現）．
  - カーネルモードとユーザモードを明確に分離（初期 Windows，Linux など）．
  - **長所**：実行性能と効率が良い．モジュールの追加で新規機能が容易に実現できる．全体機能が小さい場合は実装が比較的容易．
  - **短所**： OS が大きくなるとメモリ空間を圧迫する．カーネル機能の変更や拡張を行いにくい．カーネル自体の保護強度が弱い（障害発生時は OS 全体が停止する）．
- **マイクロカーネル**
  - プロセスに関する OS の基本操作のみをマイクロカーネルとし，他のサービス機能はカーネル外部のモジュールとして実行（各種サービスはシステムサーバ群として提供．Windows 2000/XP 以降，Mac OS X など）．
  - **長所**：モジュール化されているため機能の変更や拡張が容易．カーネルの保護強度が強く安定性が

高い．
  - 短所：プロセス間通信が性能のオーバヘッドとなる．デバイス操作もユーザモードで実行されるため監視が必要．
  - 高い信頼要件を持つシステム向けの専用 OS（工業・航空・軍事用など）では，圧倒的にこちらが採用される．

1.7 OS の起動と中断・終了

1.7.1 OS の起動手順

1. ブート ROM にあるブートストラップローダが，ハードディスクから **IPL（Initial Program Loader）** を読み出して主記憶に格納する．
2. IPL が OS プログラムをメインメモリに読み出す（この手順を**ブートストラップ / ブート**と呼ぶ）．
3. OS を初期化して各種の OS サービス機能を起動する（ハードウェアの認識や主記憶上の領域確保など）．

1.7.2 起動と再起動の種類

- **ブート：** OS を起動すること．
- **リブート：** OS の起動をやり直すこと（再起動）．
  - **コールドブート（コールドスタート）**：電源投入直後の状態からのリブート．ハードリセットであり，デバイスや OS の完全な初期化を行う．
  - **ウォームブート（ウォーム／ホットスタート）：** OS のロードと再起動からやり直すソフトウェアリセット（Ctrl+Alt+Delete による再起動など）．
- **リセット**：コンピュータが制御不能になった場合に強制的に起動し直すこと．

1.7.3 中断と終了の形態

- **スリープ（サスペンド，スタンバイ）**：省電力の待機モード．主記憶（メモリ）の内容は保持し，ハードディスクは停止させる．
- **ハイバネーション**：さらに省電力の休眠モード．主記憶の内容をハードディスクに保存して，次に起動した時に作業を再開できるようにする．
- **シャットダウン**：実行中のすべてのユーザプロセスを停止し，電源をオフにする（突然電源を切るとファイルシステムに障害が生じることもある）．

まとめ

OS の設計には、高速だが障害に弱い「モノリシックカーネル」と、通信のオーバーヘッドはあるが堅牢な「マイクロカーネル」があり、システムの要件によって使い分けられます。また、OS の起動（ブート）は小さなプログラムが段階的に大きなプログラムを呼び出す仕組みで行われ、終了や中断時はメモリやディスクの状態を適切に管理（シャットダウン・スリープ・ハイバネーション）してデータを保護します。

例え話による解説

カーネルの構成方式を「**会社の組織体制**」に例えてみましょう。

- **モノリシックカーネル（ワンマン社長の企業）**：社長（カーネル）が営業も経理もすべて直接こなします。指示の伝達が不要なので**仕事は速い（高パフォーマンス）**ですが、社長が風邪を引く（エラーが起きる）と**会社全体がストップ（システムダウン）**してしまいます。
- **マイクロカーネル（分業制の企業グループ）**：社長（カーネル）は各部署の連絡調整（プロセス間通信）だけを行い、実際の業務は独立した別部署（外部モジュール）が行います。部署間のやり取りに**手間はかかります（オーバーヘッド）**が、一つの部署が機能停止しても**会社全体への影響は最小限（高安定性）**に抑えられます。

2　プロセスの管理とスケジューリング

2.1　プロセスの管理機能

2.1.1　プロセス管理の目的と OS の役割

- **プロセス管理の定義**：プロセッサ（CPU）を時間的に制御し、実行する／実行すべきプロセスを決定して、効率良くプロセッサに割り付けることです。
- **「プロセス (process)」と「プロセッサ (processor)」**：処理の実行単位（プロセス）と、実際に処理を行うハードウェア（プロセッサ）を適切に結びつけます。
- **プロセス管理における OS の主な役割**：
  - プロセッサに対してプロセスを割り付ける。
  - 実行を開始する前に、プロセスのデータをメインメモリ（主記憶）に配置する。
  - **プロセススイッチ（プロセスディスパッチ）**：プロセッサにプロセスを割り当て、実行するプロセスを別のプロセスへ瞬時に切り替える。

2.1.2　OS のプロセス管理機能（7 つの主要機能）

1. **プロセス状態の管理**：プロセスが現在どのような状態（実行中、待機中など）にあるかを把握し管理します。
2. **プロセススイッチ**：実行しているプロセスから別のプロセスへの切り替え処理を行います。
3. **プロセスの生成と削除**：処理の開始時にプロセスを作り出し、終了時に破棄します。
4. **プロセススケジューリング**：複数のプロセスの中で、どれをどのような順序で実行するかを決定します。
5. **スレッドの生成と管理**：プロセス内のさらに細かい並行処理の単位である「スレッド」を生成・管理します。
6. **複数プロセスの同期**：複数のプロセスが共有資源を使う際に、矛盾が生じないようタイミングを制御します。
7. **複数プロセス間の通信**：独立したプロセス同士がデータをやり取りするための仕組みを提供します。

まとめ

プロセス管理の核心は、限られた資源である「プロセッサ」を多数の「プロセス」で効率的に共有することです。OS は、プロセスの生成から消滅までのライフサイクルを管理し、スケジューリングやプロセススイッチを通じて、システム全体がスムーズに動作するよう綿密に制御しています。

例え話による解説

OS のプロセス管理機能を「**会社のプロジェクト進行**」に例えてみましょう。

- **プロセスの生成と削除**：新しいプロジェクト（プロセス）を立ち上げ、完了したらチームを解散します。
- **プロセス状態の管理とスケジューリング**：各プロジェクトが「作業中」か「資材待ち」かを把握し、どの仕事から優先して進めるか（スケジュール）を決めます。
- **プロセススイッチ**：社員（プロセッサ）がプロジェクト A の作業からプロジェクト B の作業へ、頭と手元を切り替えて取り掛かります。
- **同期と通信**：複数のチームが同じ会議室（共有資源）を使う際にスケジュールを合わせ（同期）、必要な資料を共有（通信）します。

2.2 プロセスの状態と状態遷移

2.2.1 プロセスの 3 つの基本状態

- **実行可能（Ready, Runnable）**：必要なデータや資源が揃っており、プロセッサが割り当てられればいつでも実行できる状態。生成された直後のプロセスはまずこの状態になります。
- **実行（Running, Executing）**：プロセッサ（CPU）を割り当てられ、現在処理を行っている状態。1 つのプロセッサにつき、実行状態になれるプロセスは同時に 1 つのみです。
- **待ち（Waiting, Blocked）**：入出力の完了や指定時間の経過など、特定の事象の発生を待っていて実行がブロックされている状態。

2.2.2 プロセスの状態遷移（ステート遷移）

プロセスの状態は、OS の制御や自身の処理状況によって以下の 4 パターンで遷移します。

- **ディスパッチ（実行可能 → 実行）**：スケジューリングにより、実行可能状態のプロセスから 1 つが選ばれプロセッサに割り付けられます。
- **プリエンプト（実行 → 実行可能）**：OS が実行中のプロセスを強制的に中断（横取り）し、プロセッサを別のプロセスに割り当てます。
- **ブロック（実行 → 待ち）**：実行中のプロセスが、入出力などの事象発生を待つために自ら処理を中断します。
- **ウェイクアップ（待ち → 実行可能）**：待っていた事象が発生（起床指示など）し、再び実行できる状態に戻ります。

2.2.3 プロセススイッチとプロセスのライフサイクル

- **プロセススイッチ**：実行するプロセスを切り替える OS の機能です。「プロセス X → OS（割込み・スケジューリング・ディスパッチ処理を行う）→ プロセス Y」という順で制御が移ります。
- **プロセスの生成**：カーネルモードで行われます。主記憶上にプロセス領域（コード・データ領域）と PCB（プロセス制御ブロック）を確保・設定し、実行可能キューの末尾に追加してプロセスを「実行可

能状態」にします。

- **プロセスの消去**：正常・異常終了にかかわらず、PCB やプロセス領域を開放し、リストから削除して状態を消去します。異常終了時（中途消滅）は、ハードウェア資源の使用要求の強制取り下げもあわせて行われます。

まとめ

プロセスは生成されてから消滅するまで、「実行」「実行可能」「待ち」の 3 つの状態を行き来します。OS はスケジューリングによってプロセスをディスパッチ（実行開始）したり、時間切れ等でプリエンプト（強制中断）したりすることで、限られたプロセッサを効率よく回し、安全なプロセススイッチを実現しています。

例え話による解説

プロセスの状態と遷移を**「病院の診察」**に例えてみましょう。

- **実行可能（待合室）**：受付を済ませ、呼ばれればいつでも診察を受けられる患者たちです。
- **実行（診察室）**：実際に医師（プロセッサ）の診察を受けている患者です（1 つの診察室に 1 人だけ）。
- **待ち（検査結果待ち）**：血液検査などに回され、結果が出るまで診察を進められない（ブロックされた）患者です。
- **状態遷移**：待合室から診察室へ呼ばれるのが**ディスパッチ**、急患対応などで一旦診察室から出されるのが**プリエンプト**、検査結果が出て再び待合室に戻るのが**ウェイクアップ**です。

2.3 プロセス制御ブロックとその管理

2.3.1 プロセス制御ブロック（PCB）の役割と仕組み

- **プロセス制御ブロック（PCB: Process Control Block）**：プロセスの実行や状態を管理・制御するための重要な情報が格納されるデータ領域です。
- **間接的なプロセス管理**： OS はプロセスの実体そのものを直接操作するのではなく、PCB 内のポインタ（メモリ上の格納位置を示す情報）を書き換えたり操作したりすることで、間接的にプロセスを管理します。
- **実行の効率化**：重要なデータや頻繁に利用する情報を PCB に保持しておくことで、プロセスの実行中断や再開（プロセススイッチ）を高速かつ効率的に行うことができます。

2.3.2 PCB の構成情報とプロセス領域

- **PCB に格納される主な情報**：
  1. 次の PCB へのポインタ
  2. プロセスの識別番号（ID）
  3. プロセスの状態（実行可能、待ちなど）
  4. スケジューリング情報
  5. プロセス領域へのポインタ

6. **プログラムカウンタ（PC）の退避領域**：現在どこを実行しているかの情報
7. **汎用レジスタの退避領域**：プロセスが頻繁に利用するデータの格納メモリ
8. カーネル資源（メモリ、ファイル、入出力、割込み等）の管理情報
9. プロセスを使用しているユーザの情報

※ 特に (6) と (7) は、プロセスの中断・再開処理において極めて重要な役割を果たします。

- **プロセス領域（実体が格納されるメモリ領域）の構成**：
  - **コード**：プログラムの命令コード（固定、複数プロセスで共有可）
  - **データ**：読込み専用のデータ（固定、共有されることもある）
  - **スタック**：書換え可能なデータ（可変、共有されることもある）
  - **フレーム（ヒープ）**：書換え可能なデータ（可変、個々のプロセス専用領域）

2.3.3 PCB の管理手法とデータ構造

- **状態ごとの管理**：実行可能状態や待ち状態にあるプロセスの PCB は、以下のようなデータ構造（リストやキュー）を用いて実行順序が管理されます（待ち状態はキューのみで管理）。
- **管理に用いられる基本データ構造**：
  - **キュー（待ち行列）**：先入れ先出し（**FIFO**: First-In First-Out）。最後尾にデータを追加（put/enqueue）し、先頭からデータを取り出し（get/dequeue）ます。
  - **スタック（積み箱）**：先入れ後出し（**LIFO**: Last-In First-Out）。最上部にデータを追加（push）し、最上部からデータを取り出し（pop）ます。
  - **線形リスト**：データを列に連ねて格納する構造。任意の場所へのデータ追加や削除が容易で、キューやスタックを実現する基盤としても利用されます。

2.3.4 プロセスの親子関係と生成

- プロセスは自身の情報を持った別のプロセスを生成することができ、生成元を「**親**」、生成された側を「**子**」と呼びます。親子間でメモリ領域を共有することもあります。
- **UNIX におけるプロセス生成の仕組み（2 つのシステムコール）**：
  - **fork()**：自身の完全なコピーである新しいプロセスを生成します。コード領域を共有し、同じプログラムを実行します。
  - **exec()**：プロセスが指定した別のプログラムを実行できるように「変身」させます。この時点で親子間のメモリ共有は解消されます。

まとめ

PCB（プロセス制御ブロック）は、プロセスにとっての「カルテ」や「履歴書」のようなものです。OS は PCB の内容（次に実行する命令の場所や、レジスタの状態）を保存・復元することでプロセスの切り替えを行います。また、多数の PCB を「キュー」や「リスト」といったデータ構造に並べて整理することで、効率的なスケジューリングを実現しています。

例え話による解説

PCB とプロセス管理を「飲食店の厨房」に例えてみましょう。

- **プロセス領域（実体）**：調理中の鍋や食材です。コード（レシピ）、データ（調味料）、スタック/ヒープ（まな板の上の作業スペース）に分かれています。
- **PCB（注文伝票）**：「どのテーブルの注文か」「どこまで調理が進んでいるか」を記録した伝票です。OS（店長）は鍋を直接見なくても、この伝票（PCB）を見るだけで状況を把握し、指示を出せます。
- **データ構造（キューとスタック）**：
  - **キュー**：注文順に並べた伝票の列（FIFO）。先に来た注文から順に処理します。
  - **スタック**：伝票を串刺しにする注文刺し（LIFO）。一番最後に刺した（push）伝票を、一番最初に取り外し（pop）ます。
- **fork と exec**：親方（親プロセス）が弟子（子プロセス）に「俺と全く同じ作業をやれ（fork）」と指示し、その後「やっぱりお前はサラダを作れ（exec）」と別の仕事を割り当てるようなイメージです。

2.4 スレッドとその利用場面

2.4.1 スレッドの基本概念

- **スレッド（Thread）とは**：プロセスの生成や消去にかかる時間を短縮し、処理を高速化するための仕組みです。プロセスよりも身軽に動けるため、**「軽量プロセス（Lightweight Process）」**とも呼ばれます。
- **動作の仕組み**：プロセス全体（コードやデータ領域など）を丸ごと複製するのではなく、実行に必要な最低限の資源（スタックやレジスタなど）のみを複製し、それらを素早く切り替えながら実行します。

2.4.2 スレッドの主な利用場面

1. **サーバプログラムの応答性の向上**：クライアントからのリクエストが来るたびに、その 1 つのリクエストを処理するための専用スレッドを生成し、処理と応答を任せます。これにより、前の処理の完了を待つことなく、次のリクエストを並行して受け付けることができます。
2. **CPU 処理と入出力処理のオーバーラップ**：時間がかかる入出力（I/O）の完了を待機するための専用スレッドを用意することで、その待ち時間の間に、別のスレッドが CPU を使った計算などの別処理を無駄なく進めることができます。
3. **並列アルゴリズムの表現**：データが大規模で、かつ並列に動作可能なアルゴリズムを、複数のスレッドに分割して同時実行します。ただし、スレッド間でデータをやり取りする通信にはオーバーヘッド（余分な処理コスト）が掛かるため、**どんなアルゴリズムでも単純に並列化すれば実行効率が向上するわけではない**点に注意が必要です。

まとめ

プロセスが「一軒の家」だとすれば、スレッドは「家の中にいる住人」です。新しい家を建てる（プロセス生成）には多大な時間と資源（メモリ）がかかりますが、同じ家の中で住人を増やす（スレッド生成）だけであれば、リビングや冷蔵庫（コードやデータ）を共有しながら、みんなで手分けして素早く作業を進めることができます。

例え話による解説

スレッドの利用場面を**「レストランの営業」**に例えてみましょう。

- **応答性の向上**：お客さん（リクエスト）が来るたびに、手の空いているウェイター（スレッド）が個別に対応します。1 人が注文を取っている間でも、別のウェイターが次のお客さんをすぐに案内できます。
- **処理のオーバーラップ**：オーブンで肉が焼き上がるの（入出力待ち）を待っている間、シェフ（スレッド）はただ突っ立っているのではなく、並行してサラダの盛り付け（CPU 処理）を行います。
- **並列アルゴリズム**：大量の玉ねぎをみじん切りにする際、まな板と包丁（スタック等の資源）を用意して 3 人の調理補助（スレッド）で手分けして切ります。ただし、作業スペースの譲り合いや声掛け（通信のオーバーヘッド）が多くなりすぎると、かえって時間がかかることもあります。

2.5 プロセスのスケジューリング

2.5.1 スケジューリングの目的とタイミング

- **スケジューリングの必要性**： CPU のアイドル時間（遊び）を最小限に抑え、システム全体のスループット向上と実行時間の短縮を図ることです。例えば、プロセス A が入出力待ち（I/O 待ち）になった瞬間に、CPU 処理を待っているプロセス B を実行させることで、プロセッサを無駄なく稼働させます。
- **スケジューリングとは**：
  - **広義**：プロセスの実行順序を決めること全体を指します。
  - **狭義**：実行可能状態にあるプロセスから 1 つを選定し、それを実行状態にする（プロセッサを割り当てる）ことです。ディスパッチやプリエンプトの総称とも言えます。
- **スケジューリングが発生するタイミング**：
  1. 実行状態のプロセスが終了、または待ち状態へ遷移した時。
  2. あるプロセスが新しく生成されたり、待ち状態が解除されて「実行可能状態」へ遷移した時。
  3. 実行可能状態のプロセスの優先度などの情報が、アプリやユーザから明示的に更新された時。

2.5.2 マルチプログラミングと時分割制御

- **マルチプログラミング（マルチタスキング）**：時分割制御を用いてプロセスを細かく切り替えながら、プロセッサなどのハードウェア資源を複数のプロセスで共有する管理・制御方式です。プロセッサとプロセスの対応付けを「1 対多」で多重化します。
- **時分割制御（TSS: Time Sharing System）**：プロセッサの実行時間を細かく分割し、個々のプロセスに順番に割り当てて実行する方式です。この分割された数〜数十ミリ秒程度の時間単位を**「タイムス**

ライス」や「クォンタム」と呼びます。

2.5.3　プロセスの性質による分類と判定指標

プロセスは、その処理の性質によって大きく 2 つに分類されます。

- **プロセッサバウンド（CPU バウンド）**：プロセッサでの計算処理の要求が多く、入出力（I/O）要求が少ないプロセスです。実行または実行可能状態にあることが多いため、プロセッサを占有しがちになり、強制的な切替え（プリエンプト）が重要になります。
- **入出力バウンド（I/O バウンド）**：ファイル読み書きやネットワーク通信などの入出力要求が多いプロセスです。大半の時間を「待ち状態」で過ごします。
- **性質を判定する指標**：プロセッサ時間（実行状態の総時間）、入出力時間（I/O 待ちの総時間）、入出力処理量、待ち（ブロック）要因などを総合して、OS はプロセスの性質を判定します。

2.5.4　スケジューリングに関する重要用語

- **スケジューラ**：実行可能状態にあるプロセスのリストから、次に実行するべきものを「選択」する機能（モジュール）です。
- **ディスパッチャ**：スケジューラによって選択されたプロセスに対し、実際にプロセッサを「割り当てる」機能（モジュール）です。
- **スケジューリングアルゴリズム**：実行可能状態のプロセスのうち、どれを実行状態へ遷移させるか（誰を優先するか）を決定するためのルールや計算方式です。
- **多重度**：システム内に存在し、スケジューリングの対象となっているプロセスの総数のことです。
- **プロセッサ利用率（CPU 使用率）**：コンピュータの総稼働時間に対して、プロセッサが実際に有効な処理を行っていた（アイドル状態ではなかった）時間の割合です。

まとめ

スケジューリングの最大の目的は「いかに CPU を遊ばせないか（アイドル時間をなくすか）」に尽きます。CPU バウンドと I/O バウンドのプロセスを上手く混ぜ合わせ、待ち時間に別の作業をねじ込むことで、システム全体の処理能力（スループット）を最大化しています。この時分割による切り替えがあまりに高速なため、人間には複数のアプリが「同時に」動いているように見えます。

例え話による解説

スケジューリングを「**レストランの敏腕シェフ（CPU）**」に例えてみましょう。

- **CPU バウンド（ずっとかき混ぜるソース）**：つきっきりで作業が必要な料理です。シェフを占有しがちなので、焦げないように適度な時間（タイムスライス）で別の作業と切り替える必要があります。
- **I/O バウンド（オーブンで焼くグラタン）**：セットしてしまえば、焼き上がるまでシェフの手は空きます（入出力待ち）。
- **マルチプログラミングと時分割制御**：グラタンをオーブンで焼いている待ち時間の間に、ソースをかき混ぜ、その合間に別の料理の野菜を刻む……というシェフの無駄のない動きのことです。
- **スケジューラとディスパッチャ**：次にどの料理に手をつけるか頭の中で決めるのが「スケジューラ」、実際にその作業へ手を動かし始めるのが「ディスパッチャ」です。

2.6 スケジューリングアルゴリズム

2.6.1 スケジューリングの評価尺度と基準

システムの用途に応じて、以下の 5 つの定量的な尺度のどれを優先するか（最大化／最小化するか）を基準にアルゴリズムが検討されます。

- **プロセッサ利用率**：総稼働時間に対する全プロセスの動作時間の割合（目標：最大化）。
- **スループット**：単位時間あたりの完了プロセス数、仕事量（目標：最大化）。生産システムなどで特に重視されます。
- **ターンアラウンド時間**：実行要求から処理完了までの経過時間（目標：最小化）。
- **待ち時間**：実行可能または待ち状態にある時間の合計（目標：最小化）。
- **応答時間**：実行要求から「最初の応答」が開始されるまでの時間（目標：最小化）。対話システムなどで特に重視されます。

2.6.2 スケジューリング方式の分類

- **ノンプリエンプティブ（横取りなし）**：いったん実行状態になったプロセスは、自身が終了するか待ち状態になるまでプロセッサを解放しません。OS による強制的割り込みが発生しません。
- **プリエンプティブ（横取りあり）**： OS が実行中のプロセスを強制的に中断（プリエンプト）し、実行可能状態に戻して別のプロセスにプロセッサを割り当てます。

2.6.3 代表的なスケジューリングアルゴリズム

1. **到着順（FCFS: First Come First Serve）**【横取りなし】
   - **特徴**：実行可能キューに到着した順（先着順）に処理します。
   - **長所**：キューに繋ぐだけで実行順を管理でき、実現が容易です。
   - **短所**：プロセスの特性を考慮できず、実行時間の長いプロセスが先頭に来ると全体の効率が著しく悪化します。

2. **最短要求時間順（SJF: Shortest Job First）**【横取りなし】
   - **特徴**：実行時間の短いプロセスから優先して実行します。
   - **長所**：平均ターンアラウンド時間と平均待ち時間の和が最小になることが数学的に保証されます。
   - **短所**：実行する前にプロセスの実行時間が正確に分かっている必要があり、現実のシステムでの実装は困難です。
3. **ラウンドロビン**【横取りあり】
   - **特徴**：一定時間（タイムスライス／クォンタム）ごとに、実行するプロセスを固定順で強制的に切り替えます。
   - **長所**：実現が比較的容易で、すべてのプロセスに均等に時間が割り当てられるため、応答時間が短縮されます。
   - **短所**：プロセスの特性（CPU バウンドか I/O バウンドか）を考慮できません。
4. **優先度順（Priority）**【横取りあり】
   - **特徴**：制限時間やメモリ使用量などを指標にプロセスへ優先度を付与し、常に最高の優先度を持つプロセスを実行します。同一優先度同士は FCFS で処理します。
   - **長所**：プロセスの特性や重要度を考慮した柔軟なスケジューリングが可能です。
   - **短所**：優先度が低いプロセスが永遠に実行されない「**無限のブロック（飢餓状態／スタベーション）**」が発生する危険があります。
   - **解決策（エージング）**：待ち時間が長くなるにつれて、プロセスの優先度を徐々に高くしていく手法（**エージング**）でスタベーションを回避します。

2.6.4 多重レベルのスケジューリング

- **多重レベルスケジューリング**：プロセスを特性ごとに分類し、独立した複数の実行可能キューで管理して、それぞれに最適なアルゴリズムを適用する方式です。
- **多重レベルフィードバックスケジューリング**：さらに一歩進め、プロセスが実行中に別のキューへ移動することを許す方式です。例えば「長く CPU を占有したプロセスは優先度の低いキューへ落とす」「待ち時間が長いプロセスは優先度の高いキューへ格上げする」といった動的な調整を行います。

まとめ

「完璧な唯一のスケジューリングアルゴリズム」は存在しません。例えば、レスポンスの速さが命の「対話型システム」ではラウンドロビンや優先度順が適していますが、大量のデータを一括処理する「生産システム」では FCFS や SJF のようなオーバーヘッドの少ない方式が有利です。現代の OS はこれらを組み合わせた「多重レベルフィードバックスケジューリング」を採用し、状況に応じて柔軟にプロセスを捌いています。

例え話による解説

スケジューリングアルゴリズムを「スーパーのレジ打ち」に例えてみましょう。

- **到着順（FCFS）：**普通のレジです。並んだ順に会計しますが、前の人のカゴが山盛りだと後ろの人は大迷惑します。
- **最短要求時間順（SJF）：**「お買い上げ 5 点以下の方専用レジ」です。全体の平均待ち時間は減りますが、カゴ山盛りの人は一生会計できません。
- **ラウンドロビン：**全員を順番にレジに通し、「1 人 3 品スキャンしたら強制的に列の後ろに並び直させる」方式です（横取りあり）。全員が少しずつ平等に進みます。
- **優先度順とエージング：** VIP カード（優先度）を持つお客さんを常に横入りさせます。一般客がいつまでも会計できない（無限のブロック）のを防ぐため、長く待っている一般客には店長が徐々に VIP ポイントを付与し、最終的に優先してあげる仕組みが「エージング」です。

3　ネットワークとサイバーセキュリティ

3.1　サイバーセキュリティの基本概念と背景

3.1.1　サイバーセキュリティの定義と目的

サイバーセキュリティとは、コンピュータ、ネットワーク、および各種情報システムを、不正アクセス、サイバー攻撃、情報漏えい、データの破壊といった多様な脅威から保護するための技術的仕組み、運用方針、リスク管理概念、およびその継続的な実践全般を指します。

まとめ

サイバーセキュリティは単なるツール導入にとどまらず、情報資産を未知・既知の脅威から保護し、安全に運用し続けるための「概念・技術・実践の総称」です。

例え話による解説

大切な家財が詰まった自宅に「防犯カメラ」を設置したり「頑丈な二重鍵」を施したりして、空き巣や侵入者（ハッカー）の害から大切な資産を守る一連の防犯対策のようなものです。

3.1.2　サイバーセキュリティとレジリエンス（回復力）

国際電気通信連合（ITU-T）の定義等に基づき、サイバー空間の安全性は「保護」と「回復力」の双方から構成されます。

- **Cybersecurity（ITU-T 定義）**：工具（ツール）、ポリシー、セキュリティ概念、保護手段、ガイドライン、リスク管理手法、アクション、訓練、ベストプラクティス、保証、および技術の集合体。サイバー環境や組織・ユーザの資産保護を目的とします。
- **Resilience（レジリエンス / ITU-T 定義）**：セキュリティ侵害や攻撃を受けた際に、致命的な不全に陥ることなく自律的に回復・復元する能力。
- **CIA（セキュリティの 3 要素）**：
  - **Confidentiality（機密性）**：許可された者だけが情報にアクセスできること。
  - **Integrity（完全性）**：情報が改ざんされず正確・完全であること。
  - **Availability（可用性）**：必要な時に必要な情報へアクセス・利用できること。
- **コストの捉え方**: セキュリティ費用は「損失を防ぎ資産を守るための投資（守るためのコスト）」として位置づけられます。

まとめ

防御力（セキュリティ）を高めるだけでなく、侵害された際の回復力（レジリエンス）と CIA の維持をセットで考える必要があります。これらにかける費用は事業継続のための保護費用（守るためのコスト）です。

例え話による解説

高い防波堤（セキュリティ）を作って津波を防ぐだけでなく、万が一溢れてしまっても排水ポンプや迅速な復旧手順（レジリエンス）を用意して街の機能をすぐ取り戻せるように準備しておくことに似ています。

3.2 東京大学におけるセキュリティ教育

3.2.1 情報セキュリティ教育研究センター（SI センター）

東京大学におけるセキュリティ教育および研究の司令塔として「情報セキュリティ教育研究センター（SI センター）」が設置されています（Web サイト: `https://si.u-tokyo.ac.jp/`）。全学的なセキュリティ基盤の確立と人材育成を推進しています。

まとめ

SI センターは東京大学におけるサイバーセキュリティ教育・研究活動を全学的に統括し、推進する専門組織です。

例え話による解説

学校における「防災・防犯対策本部」のような存在で、キャンパス全体の安全を守るだけでなく、生徒（学生）に対する実践的な防犯講習や教育プログラムを企画・運用しています。

3.2.2 学部横断型サイバーセキュリティ教育プログラム

2022 年 4 月より、全学の 3 年生以上を対象とした「学部横断型教育プログラム」の一環として「サイバーセキュリティ教育プログラム」が導入されました。

- **プログラムの目的**: 主専攻の学問分野を問わず、実務で活用できる高度なセキュリティ知識を習得させること。
- **単位要件**:
  - 選択必修科目から **4 単位以上**
  - 選択必修科目と選択科目を合わせて計 **12 単位以上**
- **修了認定**: 履修要件を満たした修了生には「サイバーセキュリティプログラム認定証」が発行され、就職活動におけるスキル証明等に活用可能です。

まとめ

所属学部に関わらず 3 年生以上なら履修でき、合計 12 単位以上（選択必修 4 単位以上）を取得することで公式な「認定証」が得られ、就職活動時の強みとしてアピールできます。

例え話による解説

自分の主専攻という「本業の免状」とは別に、大学が公式に保証してくれる「専門ライセンス（セキュリティ資格）」を追加で獲得できる副専攻パッケージのような仕組みです。

3.3　サイバー攻撃の現状と構造

3.3.1　サイバー攻撃の影響と大規模漏えい事例

サイバー攻撃は単なるデータの損失にとどまらず、物理的な制御システムの不全（例：冷凍庫の設定温度が異常高温化するなど）を引き起こす危険性を高めています。また、不正アクセスや設定ミス、委託先経由などにより、数百万件規模の個人情報・クレジットカード情報が流出する事件が日常的に発生しています。

- **物理・システム障害**: 制御システムの改ざんによる現実空間への物理的被害。
- **多発する不正アクセス事例**: 運輸、EC、金融、医療、自治体など業界を問わず、顧客・会員情報（氏名、メアド、生年月日、クレカ情報など）が流出。
- **サプライチェーンの危険性**: 業務委託先や外部サービスの脆弱性を突かれた連鎖的な漏えいが目立っています。

まとめ

サイバー攻撃はデータ漏えいだけでなく物理的・事業的被害を引き起こします。自社だけでなく委託先やサプライチェーン全体に及ぶリスク対策が不可欠です。

例え話による解説

銀行の金庫が破られるだけでなく、家のスマート家電が乗っ取られて勝手に暖房が極熱になるような被害や、提携している配送業者のミスで自分の住所が漏れてしまうような状況です。

3.3.2　情報漏えいを引き起こす 3 つの構造的欠陥

被害が絶えない背景には、技術面・運用面に存在する「3 つの構造的欠陥」と「環境の高度化・複雑化」があります。

1. **入口の脆弱性**: 未対策の VPN 脆弱性や未適用のセキュリティパッチ、弱いパスワード認証、委託先（サプライチェーン）を経由した侵入。
2. **権限の問題**: 過剰なアクセス権限の常態化、ネットワーク分離（セグメンテーション）の不足による組織内での容易な横展開（ラテラルムーブメント）。
3. **検知の遅れ**: 恒常的な監視不足、異常検出の大幅な遅れ、アクセスログや監査証跡の不備。

さらに、レガシーシステムとクラウドの混在による管理境界の曖昧化、シャドー IT/AI の増加、委託先管理の欠如といった構造的問題がリスクを増大させています。

まとめ

侵入を防ぐ「入口対策」だけでなく、侵入後の広がりを防ぐ「権限分離」、早期に異変を察知する「監視・検知」の 3 点を網羅した多層防御が必要です。

例え話による解説

「壊れやすい玄関鍵（入口）」、「すべての部屋が開く万能鍵（権限）」、「監視カメラのない廊下（検知）」が揃った建物で、泥棒が自由に動き回れてしまうような構造です。

3.3.3 AI を活用したサイバー攻防（AI vs AI）

近年は攻撃側・防御側の双方で AI（人工知能）の活用が進み、サイバー攻防が高度化・高速化しています。

- **攻撃側の AI 利用:**
  - 脆弱性探索の全自動化（ARTEX 等のペネトレーション自動化ツール）。
  - 攻撃の圧倒的推進・高速化（わずか 43 秒での侵入事例など）。
  - 大規模スキャン（Bissa Scanner）による人間の介入なしの連続攻撃。
- **防御側の AI 活用:** 通常とは異なる挙動をリアルタイムで自動検知し、即座に対処する AI 防御システムの導入。

まとめ

攻撃者が AI を用いて全自動かつ超高速で攻撃を仕掛けてくるため、人間の手作業による防御では間に合わず、防御側も AI によるリアルタイム検知が必須となっています。

例え話による解説

手作業で鍵穴を探る泥棒ではなく「全自動で鍵孔をピッキングする超高速ロボット泥棒」に対し、「怪しい動きを即座に察知してシャッターを下ろす AI 警備ロボット」で対抗する構図です。

3.3.4 時代に伴う攻撃対象・手法の変遷

サイバー攻撃の標的は、システムの技術的進歩とともに時代を経て変化しています。

- **1990〜2000 年代（OS・アプリのバグ）**：OS やプログラムのバグを直接突く攻撃（Nimda, Sasser, Blaster など）。
- **2000〜2009 年代（Web システムの脆弱性）**：悪意ある Web サイトの閲覧やブラウザ・プラグインの弱点を突いたマルウェア感染。
- **2010 年代〜現在（エンドユーザー・人間）**：技術的対策の強化に伴い、メールの開封や詐欺、不注意など「人間の弱さ（心理）」を狙う手法へシフト。

まとめ

攻撃対象は「OS/アプリのバグ」→「Web サイト/ブラウザ」→「エンドユーザー（人間）」へと変化しており、現在では人に対するセキュリティ意識教育が極めて重要です。

例え話による解説

昔は「金庫の鍵の欠陥」を直接突いていた泥棒が、次に「お店の裏口のすき間」を狙うようになり、今では「店員を言葉巧みに騙して鍵を開けさせる」手法へシフトした流れです。

3.3.5　アカウント・認証情報を狙う攻撃手法

ユーザーの ID やパスワードを不正に入手・盗用するための代表的な攻撃手法です。

- **総当たり攻撃（Brute Force Attack）**：パスワードのあらゆる組み合わせを網羅的に試す攻撃。漏洩済みのメールアドレスや推測しやすい ID に対して全自動で試し続けます。
- **フィッシングメール / フィッシング Web**: 本物の企業やサービスを装った偽メールを送り、リンク先の偽 Web サイトへ誘導して、ユーザー自身に ID・パスワード・クレジットカード情報等を入力させ騙し取ります。

まとめ

機械的に全パターンを試す「総当たり攻撃」と、人間を騙して自発的に入力させる「フィッシング攻撃」は、どちらも認証情報を不正取得する主要な脅威です。

例え話による解説

「ダイヤル錠を片っ端からカチカチ回して解錠を試みる（総当たり）」か、「本物の銀行員になりすまして電話し、言葉巧みに暗証番号を聞き出す（フィッシング）」かの違いです。

3.3.6　身代金要求型マルウェア（ランサムウェア）

ランサムウェア（Ransomware）は、感染したコンピュータ内の文書・写真・動画・データベース等の重要なデータを強制的に暗号化し、利用不可能な状態にする悪意あるソフトウェアです。

- **仕組みと目的**: データを元に戻すための「復号鍵」と引き換えに、金銭（暗号資産など）を要求します。
- **代表的な事例**: 2017 年の「WannaCry」、2020 年の「GARMIN (WastedLocker)」、近年のサイバー犯罪集団「Qilin」による攻撃など。

まとめ

ランサムウェアは企業や個人のデータを暗号化して人質に取り、身代金を要求する極めて悪質な脅迫型サイバー攻撃です。

例え話による解説

大事な書類が入った家の金庫に勝手に別の頑丈な南京錠をかけられ、「解除してほしければ金を払え」と紙に書いて貼り付けていく脅迫行為のようなものです。

3.4　ネットワークとプロトコルの基礎

3.4.1　インターネットの概念とパケット交換方式

インターネット（The Internet）とは、限定されたエリアのネットワーク（LAN: Local Area Network）同士を相互接続し、広域ネットワーク（WAN: Wide Area Network）へと発展させた、世界規模のネットワーク相互接続体（Inter-Networking）です。

現代のインターネット通信では、伝統的な電話網のような「回線交換」ではなく「パケット交換」方式が用いられています。

- **データを小分け（パケット化）**：送信データを小さな単位（パケット）に分割して交換するノンブロッキング通信。
- **回線の共有と動的ルーティング**: 単一の物理回線を複数の通信で共有し、状況に応じて経路を動的に変更。
- **衝突（パケットクラッシュ）の発生**: 回線を共有するため通信が集中すると衝突や遅延が発生するが、柔軟に経路を変更して伝送します。

まとめ

インターネットは世界中の LAN を相互接続したものであり、データを「パケット」という小さな単位に分割して回線を共有し、動的に経路を変更しながら送受信するパケット交換方式を採用しています。

例え話による解説

1 本の線を 1 組で占有する「固定電話（回線交換）」とは違い、小荷物に分けた手紙（パケット）をトラックや道路網（共有回線）を使って、途中の混雑状況を見ながら最適な道順を選んで運ぶ「宅配便（パケット交換）」のような仕組みです。

3.4.2 プライベート IP アドレスと NAT（アドレス変換）

ネットワーク上の機器を識別するために IP アドレスが使われますが、インターネット上で利用される「グローバル IP アドレス」の枯渇を防ぐため、特定の組織内や家庭内で自由に利用できる「プライベート IP アドレス」が規定されています（RFC1918）。

- **プライベート IP アドレスの範囲 (RFC1918)**:
  - Class A: `10.0.0.0 -- 10.255.255.255`
  - Class B: `172.16.0.0 -- 172.31.255.255`
  - Class C: `192.168.0.0 -- 192.168.255.255`
- **NAT (Network Address Translation)**: プライベート IP アドレスのままではインターネットへ出られないため、ルーター等の機器がプライベート IP アドレスとグローバル IP アドレスを相互変換します。
- **ポート番号の活用（NAPT/IP マスカレード）**：ポート番号（通信アプリケーション識別番号）を紐付けることで、1 つのグローバル IP アドレスで組織内の複数機器が同時に外部と通信できます。

まとめ

社内や家庭内では RFC1918 で規定された「プライベート IP」を割り当て、外部インターネットと通信する際に「NAT」を用いてグローバル IP およびポート番号に変換して通信を識別します。

例え話による解説

大きな会社の「内線番号（プライベート IP）」のようなものです。外部に電話をかけるときは代表電話番号（グローバル IP）に変換され、交換台（NAT）がポート番号をメモしておくことで相手からの折り返し電話がどの担当者（内線）宛てかを判別します。

3.4.3　AS（自律システム）と経路制御（ルーティング）

広大なインターネットは、単一の管理者によって運用されているわけではなく、**AS（Autonomous System: 自律システム）** と呼ばれる統一された経路制御ポリシーを持つネットワークの集合体によって構成されています。

- **AS（自律システム）の特徴**: 外部からは 1 つの独立した大きなネットワークとして参照されます。各 AS には固有の「AS 番号」が割り当てられています。
  - 例: 東京大学 (AS2501)、WIDE (AS2500)、OCN (AS4713)、KDDI (AS2516)、So-net (AS2527) など。
- **2 階層の経路制御**: 経路制御（ルーティング）は「AS 内の経路制御（IGP）」と「AS 間の経路制御（EGP/BGP）」の 2 階層構造で行われます。
- **経路追跡コマンド**: パケットがどの AS やルーターを経由しているかは、`traceroute`（Linux/Mac）や `tracert`（Windows）コマンドで調べることができます。

まとめ

インターネットは各組織が運用する「AS」の組み合わせでできており、AS 内と AS 間の 2 段階で経路を制御しています。ネットワークの到達経路は `tracert` / `traceroute` コマンドで確認可能です。

例え話による解説

国際郵便に例えると「都道府県・自治体（AS）」のような単位です。自治体の内部での配送（AS 内ルーティング）と、自治体から自治体へ郵便物を引き渡す広域ルート（AS 間ルーティング）の 2 段階で宛先まで届けられます。

3.5　攻撃者によるネットワーク偵察の手法

3.5.1　ネットワーク偵察（スキャン）の手法と検証

攻撃者が標的システムへ本格的に侵入する前段階として、ターゲットに関する情報を収集する「偵察（スキャニング）」の手法と、防御側が実施する事前検証の違いについて整理します。

- **ホストスキャン**: ICMP エコー要求/応答（Ping 等）を用いて、ネットワーク上に稼働している端末（ホスト）が存在するかを確認。
- **ポートスキャン**: 目的のホストでどのポート（通信窓口）が開いており、どんなサービスが動いているかを確認。
- **OS スキャン / アプリケーションスキャン**: 通信時に返されるバナー情報や TCP/IP 通信の癖（TCP フィンガープリント）から、OS の種類や動作するソフトウェアを特定。

- **脆弱性スキャン**: システムに既知のセキュリティホール（脆弱性）が存在するかを自動的に巡回・特定。
- **ペネトレーションテストとの違い**: 脆弱性スキャナはセキュリティ上の欠陥を網羅的に「調査・列挙」するのに対し、ペネトレーションテスト（侵入テスト）は専門家が実際の手法で「システム内部まで攻略可能か」を実証検証します。

まとめ

攻撃者は「ホスト確認 → ポート確認 → OS/アプリ特定 → 脆弱性確認」の順で段階的に偵察します。脆弱性スキャンが「穴の有無の検出」であるのに対し、ペネトレーションテストは「実際に侵入できるかの実証テスト」です。

例え話による解説

空き巣が「明かりがついているか（ホスト）」、「窓が開いているか（ポート）」、「どんな鍵のメーカーか（OS/アプリ）」、「壊れやすい鍵か（脆弱性）」を下調べする流れです。ペネトレーションテストは、防犯コンサルタントが「実際に針金で開錠して侵入できるか」試す実地確認にあたります。

3.6 東京大学（UTNET）のネットワークとセキュリティ対策

3.6.1 UTNET の構成とネットワーク種別

東京大学の全学情報基盤である「UTNET（東京大学ネットワーク）」は、400Gbps の超高速回線でインターネットと接続されており、利用形態やセキュリティ要件に応じた 3 つのネットワーク区分を提供しています。

- **制限無しネットワーク**: 研究等の特殊な通信要件に対応するため、通信制限を設けていない環境。
- **個別制限ネットワーク**: 部局や研究室などのネットワーク単位で、管理者が独自のセキュリティ制限を設定・運用する環境。
- **一律制限ネットワーク（UTokyo-WiFi）**：キャンパス内の一般的なユーザーや学生が利用する Wi-Fi 網で、全学統一の安全なポリシーのもとで一律の制限を適用。

まとめ

UTNET は 400Gbps の超高速インフラを有し、セキュリティレベルと自由度に応じて「制限無し」「個別制限」「一律制限（UTokyo-WiFi）」の 3 区分で運用されています。

例え話による解説

大きな建物のセキュリティゾーンに似ています。「一般来訪者が通る手荷物検査付きエリア（UTokyo-WiFi）」、「各部署が独自の鍵で管理するエリア（個別制限）」、「特別な許可を得た研究員だけが入れる自由エリア（制限無し）」に分かれているイメージです。

3.6.2 UTM による通信制限と自動監視・通知体制

UTNET では、UTM（Unified Threat Management: 統合脅威管理）装置を活用した多層防御と、自動化された異常検知・対応体制を構築しています。

- **内 → 外（アウトバウンド制御）**：学内端末からマルウェア汚染サイトやフィッシングサイト等の危険な外部 Web サイトへの接続を自動遮断。データベースは 15 分単位の短サイクルで最新状態に更新されます。
- **外 → 内（インバウンド制御）**：NAT（アドレス変換）を活用し、外部インターネットから学内端末への不要な直接通信を原則全断（遮断）して外部からの直接攻撃を防ぎます。
- **自動監視と通知体制**: 危険な通信が検知された場合、対象の通信を即座に自動遮断。同時にシステム管理者や該当ユーザーに自動通知を行い、感染ホストの迅速な特定と調査・隔離を要請します。

まとめ

UTM によって「危険サイトへのアクセス遮断（15 分更新）」と「外部からの直接侵入遮断（NAT）」を実施し、異常発生時は自動遮断と迅速な通知・調査依頼が行われます。

例え話による解説

マンションの「高度な自動警備システム」のようなものです。外部からの不審者の進入をブロックし（外 → 内）、住民が怪しい危険スポットに行こうとすると警告して止めます（内 → 外）。万一、部屋の中で火災（感染）が発生した場合は、自動的に警報を鳴らして被害の拡大を防ぎ、管理人に緊急通知します。

3.7 関連技術（余談）

3.7.1 ダークウェブと Tor による匿名化通信

通常アクセス可能な Web サイト（サーフェスウェブ）とは異なり、専用のソフトやプロトコルを介してアクセスするオーバーレイネットワークを「ダークウェブ」と呼びます。

- **Tor (The Onion Router)**: 暗号化を重ねたデータを複数の「中継ノード」を経由させて伝送する技術。
- **IP アドレスの隠蔽**: 送信元と送信先の特定を困難にし、通信の極めて高い匿名性を実現します。

まとめ

ダークウェブは Tor などに代表されるオーバーレイネットワークであり、玉ねぎのように多重暗号化と複数ノードを経由させることで、IP アドレスや送信者を秘匿します。

例え話による解説

手紙を送る際、差出人の名前を書かずに「A さん →B さん →C さん」と複数の仲介者を挟み、それぞれが手紙を重なる封筒で包んでリレー形式で運ぶことで、誰も最終的な差出人と宛先のペアを特定できないようにする仕組みです。

3.7.2 Apple プライベートリレーによる通信保護

Apple（iPhone / iPad / Mac 等）に搭載されているプライバシー保護機能で、ユーザーのインターネット閲覧時の匿名性を高める仕組みです。

- **二重のカプセル化**: ユーザーの「IP アドレス」と「アクセス先 URL」を異なる 2 つのリレー（中継点）で分離して秘匿します。
- **プライバシー保護**: ネットワーク事業者や Apple 自身にも「誰がどのサイトを見ているか」の全体像を把握させない構造になっています。

まとめ

Apple プライベートリレーは、IP アドレスと接続先 URL を 2 箇所のリレーで分割・暗号化することで、通信内容や閲覧履歴の特定を防ぐプライバシー機能です。

例え話による解説

荷物の配送で「自分の住所だけを知っている配送業者」と「届け先だけを知っている配送業者」の 2 社を経由させ、お互いがお互いの情報を共有しないことでプライバシーを守る宅配サービスのような仕組みです。

3.7.3 TPM（Trusted Platform Module）とハードウェアセキュリティ

端末（ハードウェア）自体の安全性や暗号化キーを安全に保持・管理するための専用セキュリティチップです。

- **ハードウェアチップ**: Windows 11 で動作要件とされる「TPM 2.0」や、Mac の「Apple T2 / Apple Silicon」などが該当します。
- **主な役割**: 暗号化アルゴリズムの高速計算、暗号鍵の安全な保管、システムの改ざん検知・防止。
- **ドライブ暗号化の保護**: Windows の BitLocker や Mac の FileVault などのストレージ暗号化機能を裏側で支えています。

まとめ

TPM は暗号鍵の保管や改ざん防止を安全に行う独立したハードウェアチップであり、BitLocker や FileVault などの OS のドライブ暗号化に不可欠な役割を果たします。

例え話による解説

パソコンの中に埋め込まれた「超頑丈な金庫」のようなものです。パスワードや暗号の鍵をソフトウェアのメモリ上ではなく金庫の中に直接しまっておくため、OS が乗っ取られても鍵を直接盗み出される心配がありません。

3.8 まとめ

3.8.1 現代のサイバーセキュリティにおける課題とレジリエンス

講義全体の総括として、現代のネットワーク社会が直面している構造的リスクと必要なセキュリティマインドについて整理します。

- **不可分の関係**: ネットワークの進歩とサイバーセキュリティの脅威は背中合わせ（切っても切れない関

係）です。

- **インターネットの基盤と現状**: インターネットは元々「性善説」に基づいて設計されましたが、現在は個人レベルでも自動化ツールや AI を用いて大規模な攻撃が実行可能な時代に変化しています。
- **攻撃の分業化・商業化**: 攻撃ツールやアクセス権が地下市場で売買され、サイバー犯罪がビジネスとして成立・巧妙化しています。
- **レジリエンス（回復力）の確保**: 「完全に攻撃を防ぐ」ことだけに固執するのではなく、**「万が一侵入・受傷されても被害を最小限に抑え、迅速に復旧・継続させる能力（レジリエンス）」**の確保が極めて重要となります。

まとめ

性善説ベースのインターネットにおいて攻撃が商業化・大規模化する現在、「100% 防ぐ」ことだけでなく、「攻撃されても被害を最小限に抑えて迅速に立ち直るレジリエンス（回復力）」を確保することが最も重要です。

例え話による解説

「絶対に火事を起こさない完全な耐火建築」を目指すことだけに頼るのではなく、「万一火災が発生してもスプリンクラーで消火し、避難ルートを確保して、すぐに業務を再開できるように準備しておく（レジリエンス）」考え方へのシフトが必要です。